

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Code

```
import re

resume = """

Name: Rahul Kumar

Email: rahul.kumar@gmail.com

Mobile: 9876543210

Skills: Python, Java, SQL, Machine Learning, NLP

Experience: 3 years

"""

name = re.search(r"Name:\s*([A-Za-z ]+)", resume)
email = re.search(r"[\w.-]+@[ \w.-]+\.\w+", resume)
mobile = re.search(r"\b[6-9]\d{9}\b", resume)
experience = re.search(r"Experience:\s*(\d+)\s*years?", resume, re.I)
skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills_found = []
```

```

for skill in skills_list:

    if re.search(r"\b" + re.escape(skill) + r"\b", resume, re.I):

        skills_found.append(skill)

print("----- CANDIDATE PROFILE -----")

print("Name:", name.group(1) if name else "Not Found")

print("Email:", email.group() if email else "Not Found")

print("Mobile:", mobile.group() if mobile else "Not Found")

print("Technical Skills:", ", ".join(skills_found))

years = int(experience.group(1)) if experience else 0

print("Experience:", years, "years")

if years >= 2 and "Python" in skills_found:

    status = "Eligible"

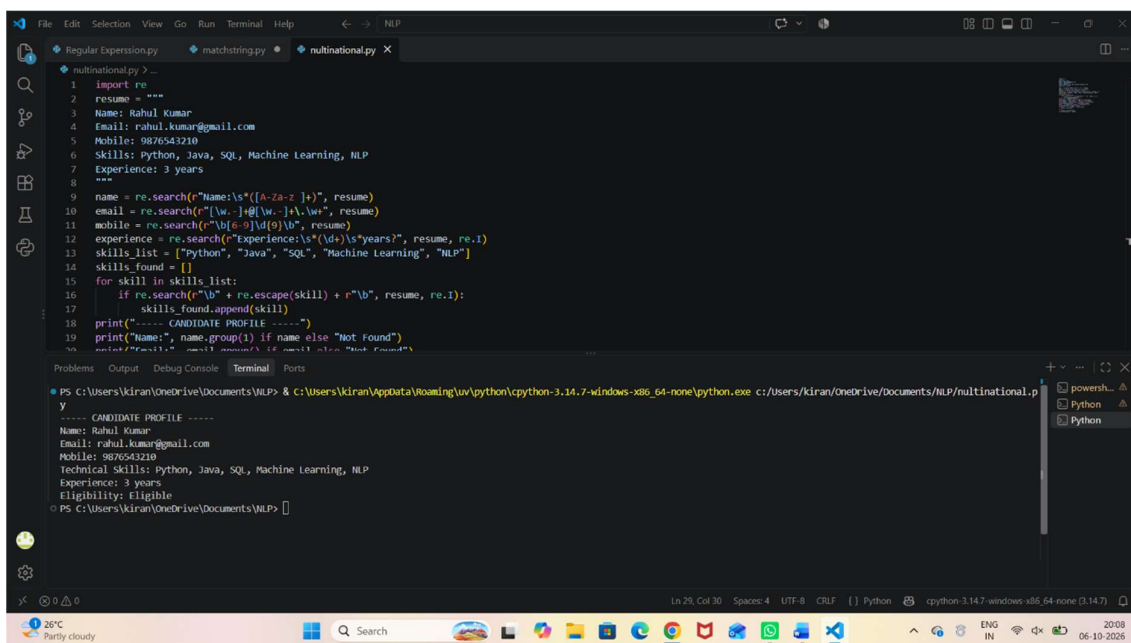
else:

    status = "Not Eligible"

print("Eligibility:", status)

```

Output



The screenshot shows a Visual Studio Code editor with a Python script named `multinational.py` open. The script uses regular expressions to parse a resume string. The output is displayed in the terminal window at the bottom.

```

1 import re
2 resume = """
3 Name: Rahul Kumar
4 Email: rahul.kumar@gmail.com
5 Mobile: 9876543210
6 Skills: Python, Java, SQL, Machine Learning, NLP
7 Experience: 3 years
8 """
9 name = re.search(r"Name:\s*([A-Za-z ]+)", resume)
10 email = re.search(r"Email:\s*([a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})", resume)
11 mobile = re.search(r"Mobile:\s*([0-9]{10})", resume)
12 experience = re.search(r"Experience:\s*(\d+)\s*years?", resume, re.I)
13 skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
14 skills_found = []
15 for skill in skills_list:
16     if re.search(r"\b" + re.escape(skill) + r"\b", resume, re.I):
17         skills_found.append(skill)
18 print("----- CANDIDATE PROFILE -----")
19 print("Name:", name.group(1) if name else "Not Found")
20 print("Email:", email.group() if email else "Not Found")
21 print("Mobile:", mobile.group() if mobile else "Not Found")
22 print("Technical Skills:", ", ".join(skills_found))
23 years = int(experience.group(1)) if experience else 0
24 print("Experience:", years, "years")
25 if years >= 2 and "Python" in skills_found:
26     status = "Eligible"
27 else:
28     status = "Not Eligible"
29 print("Eligibility:", status)

```

The terminal output shows the following results:

```

PS C:\Users\kiran\OneDrive\Documents\NLP> python multinational.py
----- CANDIDATE PROFILE -----
Name: Rahul Kumar
Email: rahul.kumar@gmail.com
Mobile: 9876543210
Technical Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
Eligibility: Eligible

```

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search

Code

```
import re

products = [
    "Python Programming Book",
    "Java Programming Book",
    "Python Laptop",
    "Java Laptop",
    "Machine Learning Course",
    "SQL Database Guide",
    "Python Course",
    "NLP Tutorial"
]

def search_products(keyword, search_type):
    if search_type == "exact":
        pattern = r"\b" + re.escape(keyword) + r"\b"
    elif search_type == "prefix":
```

```

        pattern = r"\b" + re.escape(keyword)
    elif search_type == "suffix":
        pattern = re.escape(keyword) + r"\b"
    elif search_type == "partial":
        pattern = re.escape(keyword)
    else:
        print("Invalid search type")
        return

    matches = []
    for product in products:
        if re.search(pattern, product, re.IGNORECASE):
            matches.append(product)

    print("\nSearch Keyword:", keyword)
    print("Search Type:", search_type)

    if matches:
        print("Matching Products:")
        for product in matches:
            print("-", product)
    else:
        print("No matching products found.")

    print("Total Matching Products:", len(matches))

search_products("Python", "exact")
search_products("Python", "prefix")
search_products("Book", "suffix")
search_products("Prog", "partial")
search_products("python", "partial")

```

Output

```
PS C:\Users\kiran\OneDrive\Documents\NLP> & C:\Users\kiran\AppData\Roaming\uv\python\cpython-3.14.7-windows-x86_64-none\python.exe c:/Users/kiran/OneDrive/Documents/NLP/product.py

Search Keyword: Python
Search Type: exact
Matching Products:
- Python Programming Book
- Python Laptop
- Python Course
Total Matching Products: 3

Search Keyword: Python
Search Type: prefix
Matching Products:
- Python Programming Book
- Python Laptop
- Python Course
Total Matching Products: 3

Search Keyword: Book
Search Type: suffix
Matching Products:
- Python Programming Book
- Java Programming Book
Total Matching Products: 2

Search Keyword: Prog
Search Type: partial
Matching Products:
- Python Programming Book
- Java Programming Book
Total Matching Products: 2

Search Keyword: python
Search Type: partial
Matching Products:
- Python Programming Book
- Python Laptop
- Python Course
Total Matching Products: 3
```

3.A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful

Code

```
import re

register_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course_code = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

register_pattern = r"^\d{9}$"
email_pattern = r"^\d{9}\.simats@saveetha\.com$"
course_pattern = r"^[A-Z]{3}\d{4}$"
semester_pattern = r"^[1-8]$"
mobile_pattern = r"^[1-9]\d{9}$"

register_valid = re.match(register_pattern, register_no)
email_valid = re.match(email_pattern, email)
course_valid = re.match(course_pattern, course_code)
semester_valid = re.match(semester_pattern, semester)
mobile_valid = re.match(mobile_pattern, mobile)

print("\n----- VALIDATION RESULTS -----")

if register_valid:
    print("Register Number: Valid")
else:
    print("Register Number: Invalid")

if email_valid:
    print("Institutional Email: Valid")
else:
    print("Institutional Email: Invalid")

if course_valid:
    print("Course Code: Valid")
```

else:

```
print("Course Code: Invalid")
```

if semester_valid:

```
print("Semester: Valid")
```

else:

```
print("Semester: Invalid")
```

if mobile_valid:

```
print("Mobile Number: Valid")
```

else:

```
print("Mobile Number: Invalid")
```

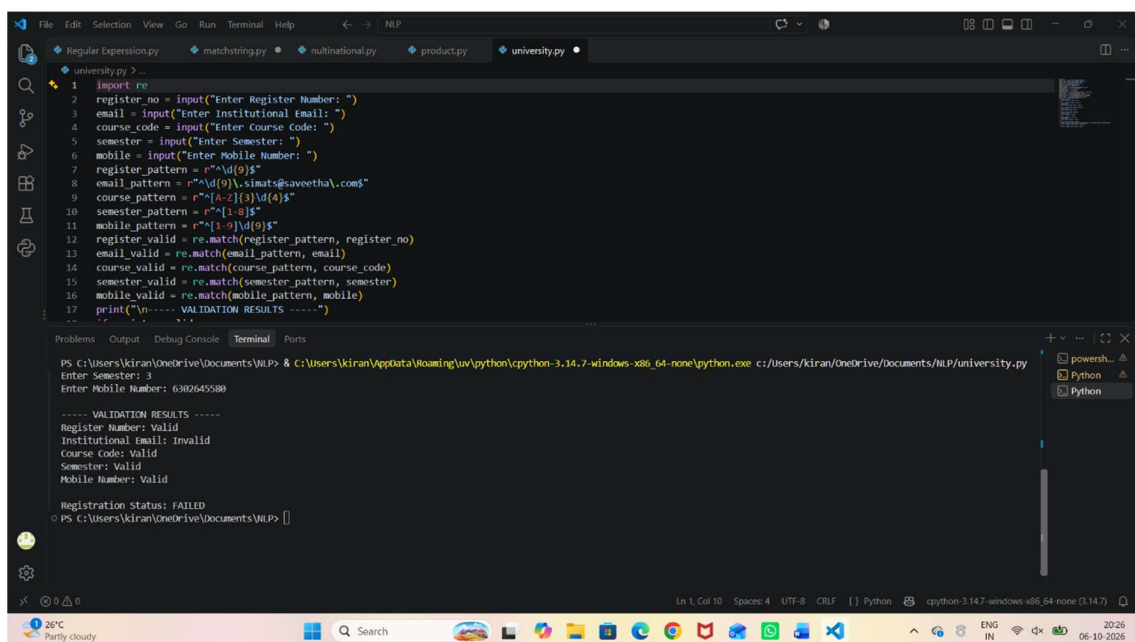
if register_valid and email_valid and course_valid and semester_valid and mobile_valid:

```
print("\nRegistration Status: SUCCESSFUL")
```

else:

```
print("\nRegistration Status: FAILED")
```

Output



The screenshot shows a Python script in a VS Code editor. The script uses regular expressions to validate user input for registration. The terminal output shows the results of running the script with specific inputs.

```
1 import re
2 register_no = input("Enter Register Number: ")
3 email = input("Enter Institutional Email: ")
4 course_code = input("Enter Course code: ")
5 semester = input("Enter Semester: ")
6 mobile = input("Enter Mobile Number: ")
7 register_pattern = r"^[0-9]$"
8 email_pattern = r"^[a-zA-Z0-9]\.simats@saveetha\.com$"
9 course_pattern = r"^[A-Z]{3}\d{4}$"
10 semester_pattern = r"^[1-8]$"
11 mobile_pattern = r"^[1-9]\d{9}$"
12 register_valid = re.match(register_pattern, register_no)
13 email_valid = re.match(email_pattern, email)
14 course_valid = re.match(course_pattern, course_code)
15 semester_valid = re.match(semester_pattern, semester)
16 mobile_valid = re.match(mobile_pattern, mobile)
17 print("\n---- VALIDATION RESULTS ----")
18
```

Terminal Output:

```
PS C:\Users\kiran\OneDrive\Documents\NLP> & C:\Users\kiran\AppData\Local\Programs\Python\Python314\python.exe c:\Users\kiran\OneDrive\Documents\NLP\university.py
Enter Semester: 3
Enter Mobile Number: 6382645588

---- VALIDATION RESULTS ----
Register Number: Valid
Institutional email: Invalid
Course code: Valid
Semester: Valid
Mobile Number: Valid

Registration Status: FAILED
PS C:\Users\kiran\OneDrive\Documents\NLP>
```